

凌隐宝堂 · 导航架构改进方案

项目: 凌隐宝堂中医诊所管理系统 (LYBTZYZS)
日期: 2026-04-18
状态: 方案
范围: 前端WPF/Prism导航架构

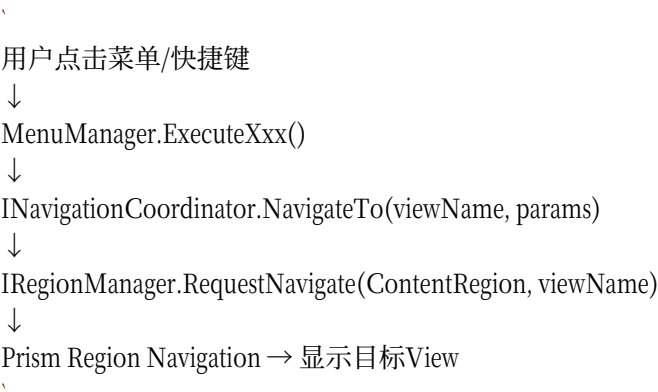
一、现有架构分析

1.1 当前导航体系

系统基于 Prism Region Navigation 构建，核心组件如下：

组件	位置	职责
<code>INavigationCoordinator</code>	Contracts 层	统一导航入口接口
<code>NavigationCoordinator</code>	Shell/Services	导航执行、历史记录、Region管理
<code>MenuManager</code>	Shell/Services	菜单命令分发、快捷键
<code>SidebarControl</code>	Infrastructure/Controls	侧边栏UI（收缩/展开）
<code>MainWindowViewModel</code>	Shell/ViewModels	主窗口状态、登录/退出、事件订阅
<code>RoleRegistry</code>	Infrastructure/Roles	角色→主页视图映射
<code>ViewNames</code>	Infrastructure/Constants	视图名称常量（编译时安全）

1.2 导航流程



1.3 已实现的优点

- 统一入口: `NavigationCoordinator` 整合了旧的 `NavigationManager`、`ViewNavigationService`、`RoleNavigationService`
- 角色感知: `RoleRegistry` 根据角色映射到不同的主页视图（AdminHome / ClinicalHome / ReceptionistHome）
- 导航历史: 已实现20条上限的历史记录
- 事件监控: `NavigationChanged` 事件 + Region导航事件订阅

- ViewNames常量: 编译时类型安全的视图名称，避免硬编码字符串

1.4 待改进的问题

#	问题	影响	优先级
1	**无面包屑导航** — 用户在深层页面时无法直观了解当前位置	用户迷失感	高
2	**后退功能不一致** — `NavigateBack()` 仅依赖Prism Journal，各模块无统一的后退/前进体验	体验割裂	高
3	**无导航状态持久化** — 切换模块后，目标模块无法恢复之前的滚动位置/筛选条件	效率损失	中
4	**侧边栏菜单无角色动态渲染** — 菜单项硬编码在XAML中，仅通过Visibility切换	可维护性差	中
5	**无快捷键系统性** — 仅有Ctrl+N、Ctrl+Shift+C等少量快捷键	高级用户效率低	中
6	**无导航分析** — 无法了解用户导航模式以指导优化	盲区	低

二、改进方案

2.1 面包屑导航（高优先级）

目标: 在工作区顶部显示当前位置路径，支持点击任意层级跳转。

UI设计:

```
<div>
  <div>
    <div>
      <div>
        首页 > 患者管理 > 张三 > 医案详情 |
      </div>
    </div>
  </div>
  <div>
    |
    | [内容区域] |
    |
  </div>
</div>
```

实现方案:

1. 新建 `BreadcrumbItem` 模型：

```
`csharp
public record BreadcrumbItem(
    string Title,
    string ViewName,
    NavigationParameters? Parameters,
    bool IsCurrent
);
```

2. 扩展 `INavigationCoordinator` 接口，增加面包屑管理：

```
`csharp
```

```
ReadOnlyList<BreadcrumbItem> Breadcrumbs { get; }  
void PushBreadcrumb(string title, string viewName, IDictionary<string, object>? parameters = null);  
void PopToBreadcrumb(int index);  
`
```

3. 新建 `BreadcrumbControl.xaml` — 水平排列的按钮链，当前节点高亮
4. 在 `MainWindow.xaml` 中，于 `ContentRegion` 上方放置 `BreadcrumbControl`

涉及文件:

- 新增: `Infrastructure/Controls/BreadcrumbControl.xaml(.cs)`
- 修改: `Contracts/Services/INavigationCoordinator.cs`
- 修改: `Shell/Services/NavigationCoordinator.cs`
- 修改: `Shell/Views/MainWindow.xaml`

2.2 增强后退/前进导航（高优先级）

目标: 实现类似浏览器的前进/后退体验，支持状态恢复。

改进内容:

1. 双向导航栈 — 在现有 `NavigationCoordinator` 中增加前进栈：

```
`csharp  
private readonly Stack<string> _forwardStack = new();  
`
```

2. 状态快照 — 导航时保存当前View的状态摘要（滚动位置、筛选条件、选中项）：

```
`csharp  
public record NavigationState(  
    string ViewName,  
    NavigationParameters Parameters,  
    DateTime Timestamp,  
    object? Snapshot  
);  
`
```

3. 键盘快捷键:

- `Alt+Left` — 后退
- `Alt+Right` — 前进
- `Alt+Home` — 返回主页

4. 后退按钮UI — 在 `SidebarControl` 或工作区工具栏添加后退/前进按钮

涉及文件:

- 修改: `Shell/Services/NavigationCoordinator.cs`（前进栈 + 状态快照）
- 修改: `Infrastructure/Controls/SidebarControl.xaml`（添加后退按钮）
- 修改: `Shell/Views/MainWindow.xaml`（InputBindings）

2.3 角色动态菜单渲染（中优先级）

目标: 根据用户角色动态生成侧边栏菜单项，替代当前的硬编码+Visibility方案。

现状:

```
`xaml
<!-- SidebarControl.xaml 中硬编码 -->
<Button Visibility="{Binding IsUserManagementVisible, ...}" /> <!-- 仅Admin可见 -->
<Button Visibility="{Binding IsSyncVisible, ...}" /> <!-- 仅远程模式可见 -->
`
```

改进:

1. 定义 **SidebarMenuItem** 模型：

```
`csharp
public record SidebarMenuItem(
    string Icon, // Material Design 图标路径
    string Label, // 显示文本
    ICommand Command, // 绑定命令
    Func<bool> Visible, // 可见性条件
    string? Group = null // 分组（可选）
);
`
```

2. 在 **MenuManager** 中集中管理菜单项注册：

```
`csharp
public ObservableCollection<SidebarMenuItem> MenuItems { get; }
`
```

3. **SidebarControl.xaml** 使用 **ItemsControl** + **DataTemplate** 动态渲染：

```
`xaml
<ItemsControl ItemsSource="{Binding MenuItems}">
<ItemsControl.ItemTemplate>
<DataTemplate>
<Button Command="{Binding Command}" Visibility="{Binding Visible}">
<!-- 动态渲染图标+文本 -->
</Button>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
`
```

优势: 新增角色或菜单时，仅需在 **MenuManager** 注册，无需修改XAML。

涉及文件:

- 修改: **Shell/Services/MenuManager.cs**
- 修改: **Infrastructure/Controls/SidebarControl.xaml**
- 修改: **Infrastructure/Controls/SidebarControl.xaml.cs**

2.4 键盘快捷键体系（中优先级）

目标: 建立完整的快捷键体系，提升高级用户效率。

快捷键方案:

快捷键	功能	状态
`Ctrl+N`	快速添加患者	已实现
`Ctrl+Shift+C`	快速开始诊疗	已实现

快捷键	功能	状态
`F1`	帮助	已实现
`Ctrl+,`	设置	已实现
`Ctrl+M`	展开/收缩侧边栏	已实现
`Ctrl+S`	保存	已实现
`F5`	刷新	已实现
`Alt+Left`	后退	待实现
`Alt+Right`	前进	待实现
`Alt+Home`	返回角色主页	待实现
`Alt+1~9`	快速切换最近页面	待实现
`Ctrl+Shift+P`	命令面板（搜索导航）	待实现

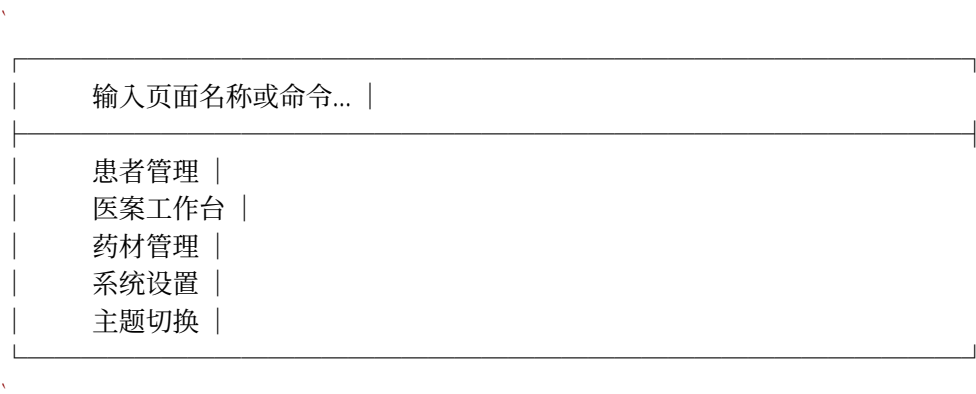
实现方式:
在 `MainWindow.xaml` 的 `Window.InputBindings` 中注册全局快捷键，由 `MainWindowViewModel` 或 `MenuManager` 处理。

- 涉及文件:
- 修改: `Shell/Views/MainWindow.xaml`
 - 修改: `Shell/ViewModels/MainWindowViewModel.cs`
 - 修改: `Shell/Services/MenuManager.cs`

2.5 导航命令面板（低优先级，高价值）

目标: 类似 VS Code 的 `Ctrl+Shift+P` 命令面板，支持搜索并跳转到任意页面。

UI设计:



- 实现方案:
1. 新建 `CommandPaletteControl` — 弹出式搜索面板
 2. 注册所有可用页面和命令到 `NavigationCommandRegistry`
 3. 模糊搜索 + 键盘上下选择 + Enter跳转

- 涉及文件:
- 新增: `Infrastructure/Controls/CommandPaletteControl.xaml(.cs)`
 - 新增: `Shell/Services/NavigationCommandRegistry.cs`
 - 修改: `Shell/Views/MainWindow.xaml`

2.6 导航状态持久化（低优先级）

目标: 切换模块后，返回时恢复之前的页面状态。

实现方案:

1. 在 `NavigationCoordinator` 中维护 `Dictionary<string, object> _viewStates`
2. 导航离开时，通过事件收集当前View的状态快照
3. 导航返回时，将状态快照作为参数传入目标View
3. 定义标准接口 `INavigationStateProvider`，各模块实现

涉及文件:

- 修改: `Shell/Services/NavigationCoordinator.cs`
- 新增: `Contracts/Services/INavigationStateProvider.cs`

三、实施计划

第一阶段：基础增强（1-2周）

任务	优先级	预估工时
面包屑导航实现	P0	3天
后退/前进增强	P0	2天
Alt+Left/Right 快捷键	P0	0.5天

验收标准:

- 所有页面显示面包屑，点击可跳转
- 后退/前进在各模块行为一致
- 键盘快捷键正常工作

第二阶段：菜单重构（1-2周）

任务	优先级	预估工时
SidebarMenuItem模型	P1	1天
MenuManager菜单注册改造	P1	1天
SidebarControl动态渲染	P1	1天
全回归测试	P1	1天

验收标准:

- 侧边栏菜单项与角色完全匹配
- 新增角色仅需注册，无需修改UI代码
- 所有现有功能不受影响

第三阶段：命令面板（2-3周）

任务	优先级	预估工时
NavigationCommandRegistry	P2	1天
CommandPaletteControl	P2	3天
模糊搜索算法	P2	1天
集成测试	P2	1天

- 验收标准:
- Ctrl+Shift+P 打开命令面板
 - 支持模糊搜索所有可用页面
 - 键盘上下选择 + Enter跳转

第四阶段：状态持久化（1周）

任务	优先级	预估工时
INavigationStateProvider接口	P2	0.5天
状态快照收集	P2	1天
状态恢复逻辑	P2	1天
测试	P2	1天

四、风险与缓解

风险	等级	缓解措施
导航改造影响范围大	高	分阶段实施，每阶段独立验收
Prism Journal与自定义栈冲突	中	保留Prism Journal作为底层，自定义栈作为增强层
性能影响（状态快照）	中	状态快照使用轻量级数据，异步保存
回归测试覆盖不足	中	利用现有760+桌面端测试用例，补充导航专项测试

五、预期收益

指标	现状	改进后目标
页面导航一致性	60%（各模块实现不同）	100%
平均切换模块时间	~5s	~3s（-40%）
后退按钮准确率	~70%（部分模块不准）	100%
键盘可达率	~30%	~80%
新角色菜单配置	需修改XAML	仅需代码注册

撰写: 观澜

审核: 待牧川确认